Reporte de Agentic Loop

0. SKILLS CONFIGURADOS

arquitecto !’ DISEÑO-ARQUITECTURA (GLX-DISEÑO-ARQUITECTURA-V1)
tech_lead !’ MICROPLANIFICACION-DISEÑO (GLX-MICROPLANIFICACION-DISEÑO-V1)
desarrollador !’ FRONTEND-DESARROLLO (GLX-FRONTEND-DESARROLLO-V1)
qa !’ test (GLX-QA-VALIDACION-V1)

1. ANÁLISIS DE ARQUITECTURA

Micro-ADR: XP-14 — Página Web Metodologías TI

1. CONTEXTO
Se requiere una página web informativa/editorial sobre metodologías de informática y TI con 3 secciones principales (Home, Historias, Noticias), una sección FAQ, aviso de cookies, y análisis de tráfico. El stack está explícitamente restringido a HTML5, CSS y JS puro. No hay backend, no hay autenticación, no hay persistencia de datos de usuario. El entorno de entrega es un archivo estático que puede servirse desde un servidor HTTP simple o CDN.

EVALUACIÓN DE ENTORNO CRÍTICA: El requerimiento especifica HTML5/CSS/JS sin mencionar servidor Node/Python/etc. Sin embargo, dado que el criterio de seguridad exige Content-Security-Policy en cabeceras HTTP e HTTPS, se asume entorno SERVIDOR (HTTP/HTTPS). Esto habilita el uso de múltiples archivos JS con `<script src>` clásico (NO type='module' para máxima compatibilidad cross-browser sin bundler). Se usará un archivo `_headers` (compatible con Netlify/Vercel/Nginx) para las cabeceras de seguridad.

2. DECISIÓN ARQUITECTÓNICA
Arquitectura elegida: Single-File-Area (SFA) — Monolito Estático Multi-Página Simulada con SPA ligera.
- Un único `index.html` con 3 secciones (`#home`, `#historias`, `#noticias`) visibles/ocultas via JS (hash routing). Esto elimina la necesidad de un servidor con routing dinámico.
- CSS dividido en 2 archivos máximo: `styles.css` (base/layout/responsive) + `components.css` (cards, FAQ, cookie banner).
- JS dividido en 3 archivos planos (sin módulos ES6 nativos): `main.js` (router + init), `security.js` (DOMPurify wrapper + sanitización), `analytics.js` (Google Analytics stub + cookie consent).
- DOMPurify se carga desde CDN con SRI hash para integridad.

3. JUSTIFICACIÓN YAGNI (Por qué NO se eligió algo más complejo)
- **NO React/Vue/Angular:** YAGNI. El contenido es mayormente estático. Un framework JS añadiría 40-200KB de bundle, tiempo de compilación y complejidad de despliegue sin ningún beneficio funcional para contenido editorial.
- **NO Next.js/Gatsby SSG:** YAGNI. No hay CMS, no hay datos dinámicos, no hay SEO dinámico que justifique un pipeline de build. HTML estático tiene mejor Time-to-First-Byte.
- **NO Microservicios / Backend API:** YAGNI absoluto. No hay datos que persistan, no hay usuarios autenticados, no hay lógica de negocio server-side requerida en los criterios de aceptación.
- **NO MongoDB/Firebase:** YAGNI. El contenido de las 3 secciones se hardcodea en el HTML o en un archivo `data/content.js` como objeto JS literal. Cero dependencias de infraestructura externa de datos.
- **NO Webpack/Vite/Parcel:** YAGNI. Sin módulos ES6 nativos, los `<script>` clásicos concatenados en orden son suficientes y funcionan en cualquier entorno sin pipeline de build.
- **NO type='module':** Aunque el entorno es servidor, se evita para garantizar compatibilidad con Safari antiguo y eliminar la necesidad de un bundler o servidor con MIME types correctos configurados.

4. RIESGOS ASUMIDOS POR SIMPLIFICAR
- **Riesgo 1 (BAJO):** El contenido hardcodeado en `data/content.js` requiere un desarrollador para actualizar noticias/historias. Mitigación aceptada: el requerimiento no menciona CMS ni panel de administración.
- **Riesgo 2 (BAJO):** Sin bundler, el orden de carga de scripts es manual. Mitigación: scripts cargados con `defer` en orden explícito al final del `<body>`.
- **Riesgo 3 (MEDIO):** Google Analytics requiere consentimiento de cookies (GDPR/criterio 7). Mitigación:

`analytics.js` inicializa GA solo después de aceptación explícita del banner de cookies.
- **Riesgo 4 (BAJO):** DOMPurify desde CDN introduce dependencia externa. Mitigación: se usa SRI (Subresource Integrity) hash para garantizar integridad del script.

5. DECISIONES DE SEGURIDAD (Criterios de Aceptación — Seguridad)
- **XSS:** DOMPurify sanitiza cualquier contenido dinámico insertado via innerHTML. Todo contenido estático usa textContent o está hardcodeado.
- **CSP:** Archivo `_headers` define `Content-Security-Policy` restrictiva: `default-src 'self'; script-src 'self' https://cdn.jsdelivr.net https://www.googletagmanager.com 'nonce-{generado}'; style-src 'self' 'unsafe-inline'; img-src 'self' https://placehold.co data:;`
- **HTTPS:** Responsabilidad del hosting (Netlify/Vercel/Nginx). El archivo `_headers` incluye `Strict-Transport-Security`.
- **Cookie Consent:** Banner implementado en `analytics.js` que bloquea GA hasta aceptación. Estado persistido en `localStorage`.

2. MÉTRICAS DE ITERACIÓN (QA)

- index.html | Estado: APROBADO | Iteraciones: 1
- assets/js/data/content.js | Estado: APROBADO | Iteraciones: 1
- assets/js/security.js | Estado: APROBADO | Iteraciones: 1
- assets/js/analytics.js | Estado: APROBADO | Iteraciones: 1
- assets/js/main.js | Estado: APROBADO | Iteraciones: 1
- assets/css/styles.css | Estado: APROBADO | Iteraciones: 1
- assets/css/components.css | Estado: APROBADO | Iteraciones: 1
- _headers | Estado: APROBADO | Iteraciones: 1
- robots.txt | Estado: SKIPPED (doc) | Iteraciones: 0

3. MÉTRICAS DE TOKENS IA

Tokens de entrada (input): 113,868
Tokens de salida (output): 45,599
Total tokens: 159,467
Modelo utilizado: claude-sonnet-4-6

Ø=Üh Ø=Ü» ~32.3 horas-hombre ahorradas
"H squad de 4 personas × 4.1 días

Desglose por Agente:

DISEÑO-ARQUITECTURA: !“910 input | !‘1,854 output | 1 llamadas | ~1.5h ahorradas
MICROPLANIFICACION-DISEÑO: !“33,901 input | !‘20,116 output | 9 llamadas | ~15.1h ahorradas
FRONTEND-DESARROLLO: !“50,646 input | !‘14,511 output | 8 llamadas | ~8.6h ahorradas
test: !“28,411 input | !‘9,118 output | 9 llamadas | ~7.1h ahorradas